

Generative Models in Finance

Week 5: Flow Matching and Diffusion Models — Training and Applications

Cristopher Salvi
Imperial College London

Spring 2026

Part 5: Training — Flow Matching

- Consider a flow model: $X_0 \sim p_{\text{init}}, dX_t = u_t^\theta(X_t) dt$
- We want $u_t^\theta \approx u_t^{\text{target}}$. A natural objective is the **flow matching loss**:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2 \right] \quad (4.1)$$

- **Problem:** We don't know $u_t^{\text{target}}(x)$ — it involves an intractable integral (??)
- **Idea:** Replace the marginal target with the *conditional* target $u_t^{\text{target}}(x | z)$, which we know analytically

The Conditional Flow Matching Loss

- Define the **conditional flow matching (CFM)** loss:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.2)$$

- Note: we regress against the *conditional* vector field $u_t^{\text{target}}(x | z)$ instead of the marginal $u_t^{\text{target}}(x)$
- This loss is *tractable*: sample $t \sim \text{Unif}[0, 1]$, $z \sim p_{\text{data}}$, $x = \alpha_t z + \beta_t \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$ (i.e. $x \sim p_t(\cdot | z)$, which is Gaussian), and compute $\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2$
- But does minimising $\mathcal{L}_{\text{CFM}}$ also minimise $\mathcal{L}_{\text{FM}}$?

Equivalence of Flow Matching Losses

Theorem 4.1

The marginal and conditional flow matching losses are equal up to a constant independent of θ :

$$\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C,$$

where C does not depend on θ . Therefore $\nabla_{\theta}\mathcal{L}_{\text{FM}}(\theta) = \nabla_{\theta}\mathcal{L}_{\text{CFM}}(\theta)$.

- **Consequence:** Minimising the tractable $\mathcal{L}_{\text{CFM}}$ with SGD is equivalent to minimising the intractable $\mathcal{L}_{\text{FM}}$

Proof of Loss Equivalence

Expand $\mathcal{L}_{\text{FM}}$ by completing the square:

$$\begin{aligned}\mathcal{L}_{\text{FM}}(\theta) &\stackrel{(i)}{=} \mathbb{E}_{t, x \sim p_t} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x) + \|u_t^{\text{target}}(x)\|^2 \right] \\ &\stackrel{(ii)}{=} \mathbb{E}_{t, x \sim p_t} \left[\|u_t^\theta(x)\|^2 \right] - 2\mathbb{E}_{t, x \sim p_t} \left[u_t^\theta(x)^T u_t^{\text{target}}(x) \right] + C_1\end{aligned}$$

where (i) is $\|a - b\|^2 = \|a\|^2 - 2a^T b + \|b\|^2$, (ii) defines $C_1 = \mathbb{E}[\|u_t^{\text{target}}(x)\|^2]$ (independent of θ).

By the law of total probability, $p_t(x) = \int p_t(x | z) p_{\text{data}}(z) dz$, so $\mathbb{E}_{x \sim p_t}[\cdot] = \mathbb{E}_{z \sim p_{\text{data}}, x \sim p_t(\cdot|z)}[\cdot]$. Applying this:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 \right] - 2\mathbb{E}_{t, x \sim p_t} \left[u_t^\theta(x)^T u_t^{\text{target}}(x) \right] + C_1$$

Proof of Loss Equivalence (cont.)

The key step: rewrite the cross term using the marginal vector field identity:

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

$$\begin{aligned} \mathbb{E}_{t, x \sim p_t} [u_t^\theta(x)^T u_t^{\text{target}}(x)] &\stackrel{(i)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T \left[\int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \right] dx dt \\ &\stackrel{(ii)}{=} \int_0^1 \int \int u_t^\theta(x)^T u_t^{\text{target}}(x | z) p_t(x | z) p_{\text{data}}(z) dz dx dt \\ &\stackrel{(iii)}{=} \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} [u_t^\theta(x)^T u_t^{\text{target}}(x | z)] \end{aligned}$$

where (i) substitutes the definition of $u_t^{\text{target}}(x)$ from the identity above; (ii) cancels $p_t(x)$ (valid since $p_t(x) > 0$ on the support of the integrand) and applies Fubini–Tonelli (the integrand is square-integrable under our regularity assumptions); (iii) recognises the integrand $p_t(x | z) p_{\text{data}}(z)$ as the joint density of $z \sim p_{\text{data}}, x \sim p_t(\cdot | z)$, so the integral is their joint expectation. This is the crucial step: the intractable marginal vector field is replaced by the tractable conditional one.

Proof of Loss Equivalence (cont.)

Substituting back:

$$\begin{aligned}\mathcal{L}_{\text{FM}}(\theta) &= \mathbb{E}_{t,z,x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) \right] + C_1 \\ &= \mathbb{E}_{t,z,x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) \right. \\ &\quad \left. + \|u_t^{\text{target}}(x|z)\|^2 - \|u_t^{\text{target}}(x|z)\|^2 \right] + C_1 \\ &= \underbrace{\mathbb{E}_{t,z,x} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2 \right]}_{=\mathcal{L}_{\text{CFM}}(\theta)} + \underbrace{C_1 - \mathbb{E} \left[\|u_t^{\text{target}}(x|z)\|^2 \right]}_{=:C}\end{aligned}$$

Since C is independent of θ , we have $\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C$. □

Conditional Optimal Transport (CondOT) Paths

- Recall: for Gaussian probability paths $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, we need to choose α_t and β_t with boundary conditions $\alpha_0 = 0, \beta_0 = 1$ (pure noise at $t = 0$) and $\alpha_1 = 1, \beta_1 = 0$ (data at $t = 1$)

- The **CondOT** choice is:

$$\alpha_t = t, \quad \beta_t = 1 - t$$

- The conditional path is $p_t(\cdot | z) = \mathcal{N}(tz, (1 - t)^2 I_d)$, and a sample is:

$$x = tz + (1 - t)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I_d)$$

- Why “optimal transport”?** Conditional on z , the path $x_t = (1 - t)\epsilon + tz$ is a *constant-speed straight line* from ϵ to z — this is the displacement interpolation from OT theory, and it minimises the kinetic energy $\int_0^1 \|\dot{x}_t\|^2 dt$
- Other valid schedules (e.g. VP/VE from diffusion models) also satisfy the boundary conditions, but move mass along curved or non-uniform-speed paths, resulting in more complex vector fields
- Consequence:** CondOT paths do not cross and the conditional vector field $u_t^{\text{target}}(x | z) = z - \epsilon$ is constant in t , making it easier for the network to learn

Flow Matching for Gaussian CondOT Paths

- Recall the general conditional Gaussian vector field (Week 4):

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$$

- For the **CondOT path**: $\alpha_t = t$, $\beta_t = 1 - t$, so $\dot{\alpha}_t = 1$, $\dot{\beta}_t = -1$:

$$u_t^{\text{target}}(x | z) = \left(1 - \frac{-1}{1-t} \cdot t \right) z + \frac{-1}{1-t} x = \frac{1}{1-t} z - \frac{1}{1-t} x$$

- With $x = \alpha_t z + \beta_t \epsilon = tz + (1-t)\epsilon$, substitute:

$$u_t^{\text{target}}(x | z) = \frac{z - (tz + (1-t)\epsilon)}{1-t} = \frac{(1-t)(z - \epsilon)}{1-t} = z - \epsilon$$

- The CFM loss becomes:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\| u_t^\theta(tz + (1-t)\epsilon) - (z - \epsilon) \|^2 \right] \quad (4.3)$$

- Remarkable simplicity**: Sample data z , noise ϵ , time t ; interpolate $x = tz + (1-t)\epsilon$; regress $u_t^\theta(x)$ against $z - \epsilon$
- This is used by *Stable Diffusion 3*, *Meta Movie Gen Video*, and many other state-of-the-art models

Flow Matching Training Algorithm

Algorithm 3: Flow Matching Training (Gaussian CondOT)

Require: Dataset $z \sim p_{\text{data}}$, neural network u_t^θ

1. For each mini-batch do:
2. Sample z from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x = tz + (1 - t)\epsilon$
6. Compute loss $\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2$
7. Update θ via gradient descent on $\mathcal{L}(\theta)$

For general Gaussian paths $p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$: set $x = \alpha_t z + \beta_t \epsilon$ and regress against $u_t^{\text{target}}(x \mid z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x$.

Score Matching

- Recall the SDE extension (Week 4): $dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$ requires knowing the **marginal score** $\nabla \log p_t$
- To use this in practice, we need to approximate the score with a neural network
- We train a **score network** $s_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ via the **score matching loss**:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} \left[\|s_t^\theta(x) - \nabla \log p_t(x)\|^2 \right] \quad (4.4)$$

- Again intractable (we don't know $\nabla \log p_t(x)$). Define the **conditional score matching loss**:

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.5)$$

Equivalence of Score Matching Losses

Theorem 4.2

The score matching loss equals the conditional score matching loss up to a constant:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathcal{L}_{\text{CSM}}(\theta) + C,$$

where C is independent of θ .

- The proof is *identical* to the flow matching case (Theorem 4.1), with the substitutions:

$$u_t^\theta(x) \rightsquigarrow s_t^\theta(x), \quad u_t^{\text{target}}(x) \rightsquigarrow \nabla \log p_t(x), \quad u_t^{\text{target}}(x | z) \rightsquigarrow \nabla \log p_t(x | z)$$

- The only ingredient to check: the marginal score admits the same mixture decomposition as the marginal vector field:

$$\nabla \log p_t(x) = \int \nabla \log p_t(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz$$

This holds because

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)} = \frac{1}{p_t(x)} \int \nabla p_t(x | z) p_{\text{data}}(z) dz = \int \nabla \log p_t(x | z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz$$

- Given this, the complete-the-square argument and the cross-term cancellation carry over verbatim. □

Denoising Score Matching for Gaussian Paths

- For Gaussian paths $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional score is $\nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2} = -\frac{\epsilon}{\beta_t}$. Substituting:

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{t,z,\epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| s_t^\theta(\alpha_t z + \beta_t \epsilon) + \frac{\epsilon}{\beta_t} \right\|^2 \right] \quad (4.6)$$

- The network learns to predict the noise ϵ that was used to corrupt z — this is **denoising score matching**
- Alternatively, reparameterise via a **noise predictor** $\epsilon_t^\theta(x) := -\beta_t s_t^\theta(x)$:

$$\mathcal{L}_{\text{DDPM}}(\theta) = \mathbb{E}_{t,z,\epsilon} \left[\left\| \epsilon_t^\theta(\alpha_t z + \beta_t \epsilon) - \epsilon \right\|^2 \right] \quad (4.7)$$

- This is the loss used in **Denoising Diffusion Probabilistic Models** (DDPM)¹

¹Ho, J., Jain, A. & Abbeel, P. *Denoising Diffusion Probabilistic Models*, NeurIPS 2020, arXiv:2006.11239.

Score Matching Training Algorithm

Algorithm 4: Score Matching Training (Gaussian paths)

Require: Dataset $z \sim p_{\text{data}}$, score network s_t^θ or noise predictor ϵ_t^θ

1. **For** each mini-batch **do**:
2. Sample z from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x_t = \alpha_t z + \beta_t \epsilon$
6. Compute loss $\mathcal{L}(\theta) = \|s_t^\theta(x_t) + \epsilon/\beta_t\|^2$
 (Alternatively: $\mathcal{L}(\theta) = \|\epsilon_t^\theta(x_t) - \epsilon\|^2$)
7. Update θ via gradient descent on $\mathcal{L}(\theta)$

Conversion Between Vector Field and Score

Proposition 4.3 (Conversion formula for Gaussian probability paths)

For $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional (resp. marginal) vector field and score are related by:

$$u_t^{\text{target}}(x | z) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x, \quad (4.8)$$

$$u_t^{\text{target}}(x) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x) + \frac{\dot{\alpha}_t}{\alpha_t} x. \quad (4.9)$$

The vector field $u_t^{\text{target}}(x)$ in (4.9) is called the *probability flow ODE*.

Proof of the Conversion Formula

Recall: $u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x$ and $\nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}$.

$$\begin{aligned} u_t^{\text{target}}(x | z) &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \\ &\stackrel{(i)}{=} \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \left(\frac{\alpha_t z - x}{\beta_t^2} \right) + \frac{\dot{\alpha}_t}{\alpha_t} x \\ &= \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x \end{aligned}$$

Step (i): write $z = \frac{x}{\alpha_t} + \frac{\alpha_t z - x}{\alpha_t}$ and collect terms (valid for $\alpha_t \neq 0$, i.e. $t > 0$).

Proof of the Conversion Formula (cont.)

To pass from the *conditional* to the *marginal* identity: integrate both sides against $\frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz$.

- LHS yields $u_t^{\text{target}}(x)$ by the marginal vector field identity
- RHS: the score term yields $\nabla \log p_t(x)$ by the marginal score identity; the $\frac{\dot{\alpha}_t}{\alpha_t} x$ term passes through (it does not depend on z , and $\int \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz = 1$)

This gives the marginal conversion formula. □

Consequence: For Gaussian paths, learning either u_t^θ or s_t^θ suffices — they are interconvertible. No need to train both separately.

Sampling with Both Networks

- Given a trained score s_t^θ and vector field u_t^θ (or one derived from the other), sample via the SDE for any $\sigma_t \geq 0$:

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[u_t^\theta(X_t) + \frac{\sigma_t^2}{2} s_t^\theta(X_t) \right] dt + \sigma_t dW_t \quad (4.10)$$

- $\sigma_t = 0$: **deterministic** sampling (flow/probability flow ODE)
- $\sigma_t > 0$: **stochastic** sampling (diffusion)
- In practice, the optimal σ_t is found empirically (balances numerical and training errors)
- Using the conversion formula with a single network:

$$dX_t = \left[\left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t + \frac{\sigma_t^2}{2} \right) s_t^\theta(X_t) + \frac{\dot{\alpha}_t}{\alpha_t} X_t \right] dt + \sigma_t dW_t \quad (4.11)$$

Summary: Training

Summary

Flow Matching: Train u_t^θ via the conditional flow matching loss:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2 \right].$$

Score Matching: Train s_t^θ via conditional score matching:

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x | z)\|^2 \right].$$

For Gaussian paths, these are equivalent (up to reparameterisation). After training, sample via ODE or SDE.

Part 6: Literature Guide — Discrete vs. Continuous Time

- The first denoising diffusion models^{2,3,4} used **discrete-time** Markov chains with time steps $t = 0, 1, 2, 3, \dots$
- The loss function was approximated via an **evidence lower bound (ELBO)** — only a lower bound on the true log-likelihood
- Song et al.⁵ showed these constructions are essentially approximations of *continuous-time* SDEs
- In continuous time, the ELBO becomes *tight*: Theorem 4.1 and Theorem 4.2 are equalities (not lower bounds)
- **Advantage of continuous time**: Mathematically cleaner; one can control the simulation error via ODE/SDE samplers post-training; no need to commit to a time discretisation before training

²Sohl-Dickstein, J. et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*, ICML 2015, arXiv:1503.03585.

³Ho, J. et al. *Denoising Diffusion Probabilistic Models*, NeurIPS 2020, arXiv:2006.11239.

⁴Song, Y. & Ermon, S. *Generative Modeling by Estimating Gradients of the Data Distribution*, NeurIPS 2019, arXiv:1907.05600.

⁵Song, Y. et al. *Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021, arXiv:2011.13456.

Forward Processes and Probability Paths

- Early diffusion model papers did not use the term “probability path”. Instead, they constructed a **forward process** — an SDE that gradually *noises* the data:

$$\bar{X}_0 = z, \quad d\bar{X}_t = u_t^{\text{forw}}(\bar{X}_t) dt + \sigma_t^{\text{forw}} d\bar{W}_t \quad (4.12)$$

- Starting from data $z \sim p_{\text{data}}$, the forward process converges to $\mathcal{N}(0, I_d)$ for $t \rightarrow \infty$
- The conditional distribution $\bar{X}_t \mid \bar{X}_0 = z$ is a conditional probability path $\bar{p}_t(\cdot \mid z)$
- For affine drift $u_t^{\text{forw}}(x) = a_t x$, the conditional distribution is Gaussian:

$$\bar{X}_t \mid \bar{X}_0 = z \sim \mathcal{N}(\alpha_t z, \beta_t^2 I), \quad \alpha_t = \exp\left(\int_0^t a_r dr\right), \quad \beta_t^2 = \alpha_t^2 \int_0^t \frac{(\sigma_r^{\text{forw}})^2}{\alpha_r^2} dr.$$

- **Key point:** A forward process is a *specific way* of constructing a Gaussian probability path. The term “probability path” (introduced by flow matching) is more general and simpler
- **Important:** Forward processes use an **inverted time convention**: $\bar{p}_0(\cdot \mid z) = \delta_z$ (data at $t = 0$, noise at $t = \infty$), opposite to our convention

Time-Reversal vs. Fokker–Planck

- The original diffusion model literature derived the training target via the **time-reversal** of the forward process⁶
- A **time-reversal** $(X_t)_{0 \leq t \leq T}$ of $(\bar{X}_t)$ is an SDE with the same distribution over trajectories but with time running backwards:

$$\mathbb{P}[\bar{X}_{t_1} \in A_1, \dots, \bar{X}_{t_n} \in A_n] = \mathbb{P}[X_{T-t_1} \in A_1, \dots, X_{T-t_n} \in A_n]$$

- Anderson's formula:

$$dX_t = \left[-u_t(X_t) + \sigma_t^2 \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$$

where $u_t(x) = u_{T-t}^{\text{forw}}(x)$ and $\sigma_t = \sigma_{T-t}^{\text{forw}}$

- This is a specific instance of the SDE extension theorem from Week 4 (after accounting for time inversion)
- **Modern approach** (as in this course): Derive the training target directly via the Fokker–Planck/continuity equation, avoiding the need for time-reversal entirely. This was pioneered by flow matching^{7,8} and stochastic interpolants⁹

⁶Anderson, B.D.O. *Reverse-time diffusion equation models*, Stochastic Processes and their Applications, 12(3):313–326, 1982.

⁷Lipman, Y. et al. *Flow Matching for Generative Modeling*, ICLR 2023, arXiv:2210.02747.

⁸Liu, X. et al. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*, ICLR 2023, arXiv:2209.03003.

⁹Albergo, M.S., Boffi, N.M. & Vanden-Eijnden, E. *Stochastic Interpolants: A Unifying Framework for Flows and Diffusions*, arXiv:2303.08797, 2023.

Flow Matching and Stochastic Interpolants

- **Flow matching**¹⁰: Restricts to ODEs (i.e. $\sigma_t = 0$). Key contribution: showing that one does *not* need a forward process or time-reversal to train generative models scalably
- **Stochastic interpolants**¹¹: Extends flow matching to include the SDE case via the “Langevin dynamics” extension (Week 4, SDE extension theorem)
- Both frameworks use probability paths instead of forward processes, and construct training targets via the Fokker–Planck equation
- **Advantages over classical diffusion:**
 - ▶ Simpler training framework (direct MSE regression)
 - ▶ Greater generality: arbitrary p_{init} (not just Gaussian) and arbitrary probability paths (not just Gaussian)
 - ▶ Classical denoising diffusion models are recovered as a special case (Gaussian initial distribution + Gaussian probability path)

¹⁰Lipman et al. (2023), arXiv:2210.02747; Liu et al. (2023), arXiv:2209.03003.

¹¹Albergo, Boffi & Vanden-Eijnden (2023), arXiv:2303.08797.

Part 7: Building an Image Generator — Guidance

- So far: unconditional generation ($X_1 \sim p_{\text{data}}$)
- Now: generate objects **conditioned on** a variable $y \in \mathcal{Y}$ (e.g. text prompt, class label)
- Goal: sample from $p_{\text{data}}(\cdot | y)$ for any y
- We use the term **guided** (rather than “conditional”) to avoid confusion with “conditional probability path” $p_t(\cdot | z)$
- A **guided diffusion model** has a vector field $u_t^\theta(\cdot | y) : \mathbb{R}^d \times \mathcal{Y} \times [0, 1] \rightarrow \mathbb{R}^d$ and samples via:

$$X_0 \sim p_{\text{init}}$$

$$dX_t = u_t^\theta(X_t | y) dt + \sigma_t dW_t$$

$$\text{Goal: } X_1 \sim p_{\text{data}}(\cdot | y)$$

Guided Flow Matching

- For fixed y , use $p_{\text{data}}(\cdot | y)$ as the data distribution and apply CFM:

$$\mathcal{L}_{\text{CFM}}^{\text{guided}}(\theta) = \mathbb{E}_{(z,y) \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x | y) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.13)$$

- Note: the conditional probability path $p_t(\cdot | z)$ and target $u_t^{\text{target}}(x | z)$ do *not* depend on y — only the network input does
- In practice: the dataloader returns pairs (z, y)
- **Empirical observation:** This procedure is theoretically correct but often produces images that don't fit the prompt y well enough. The network learns $p_{\text{data}}(x | y)$ faithfully, but:
 - ▶ The conditional distribution $p_{\text{data}}(x | y)$ is broad — many images are consistent with a given prompt, so the model tends to produce “average” outputs
 - ▶ The network may under-condition on y , especially when the text embedding carries limited information or training data for that prompt is scarce
 - ▶ Users typically want samples from a “sharpened” version of $p_{\text{data}}(x | y)$ that places more mass on images strongly matching y

Classifier-Free Guidance: Derivation

- For Gaussian paths, using Proposition 4.3, the guided vector field can be rewritten as:

$$u_t^{\text{target}}(x | y) = a_t x + b_t \nabla \log p_t(x | y) \quad (4.14)$$

for explicit $(a_t, b_t) = \left(\frac{\dot{\alpha}_t}{\alpha_t}, \frac{\dot{\alpha}_t \beta_t^2 - \dot{\beta}_t \beta_t \alpha_t}{\alpha_t} \right)$

- By Bayes' rule:

$$\nabla \log p_t(x | y) = \nabla \log p_t(x) + \nabla \log p_t(y | x) \quad (4.15)$$

since $\nabla \log p_t(y) = 0$ w.r.t. the gradient in x

- So we can decompose the guided vector field as:

$$u_t^{\text{target}}(x | y) = \underbrace{a_t x + b_t \nabla \log p_t(x)}_{\text{unconditional: } u_t^{\text{target}}(x)} + \underbrace{b_t \nabla \log p_t(y | x)}_{\text{"how much } y \text{ shifts the score"}}$$

- The term $\nabla \log p_t(y | x)$ measures how the prompt y modifies the unconditional score — it pushes x towards regions where y is more likely given x
- Idea:** At $w = 1$ we recover the true conditional model, but empirically this under-conditions on y . Amplify the prompt-dependent term by a **guidance scale** $w > 1$:

$$\tilde{u}_t(x | y) = u_t^{\text{target}}(x) + w b_t \nabla \log p_t(y | x)$$

- This is equivalent to sampling from a “sharpened” distribution $\tilde{p}(x | y) \propto p(x) p(y | x)^w$, which concentrates more mass on images strongly matching y

Classifier-Free Guidance: The Formula

Substitute $b_t \nabla \log p_t(y | x) = b_t \nabla \log p_t(x | y) - b_t \nabla \log p_t(x)$ (Bayes) into $\tilde{u}_t(x | y) = u_t^{\text{target}}(x) + w b_t \nabla \log p_t(y | x)$:

$$\begin{aligned}\tilde{u}_t(x | y) &= a_t x + b_t \nabla \log p_t(x) + w b_t \nabla \log p_t(x | y) - w b_t \nabla \log p_t(x) \\ &= a_t x + (1 - w) b_t \nabla \log p_t(x) + w b_t \nabla \log p_t(x | y) \\ &= (1 - w) \underbrace{(a_t x + b_t \nabla \log p_t(x))}_{u_t^{\text{target}}(x)} + w \underbrace{(a_t x + b_t \nabla \log p_t(x | y))}_{u_t^{\text{target}}(x|y)}\end{aligned}$$

Classifier-Free Guidance (CFG)

$$\tilde{u}_t(x | y) = (1 - w) u_t^{\text{target}}(x) + w u_t^{\text{target}}(x | y) \quad (4.16)$$

- $w = 0$: unconditional generation $w = 1$: true conditional $w > 1$: amplified guidance
- Only requires two networks (or one with label dropping) — no separate classifier needed

CFG Training

- The CFG formula requires both the conditional $u_t^{\text{target}}(x | y)$ and the unconditional $u_t^{\text{target}}(x)$
- **Key insight:** Train a *single* network $u_t^\theta(x | y)$ that learns both, via **label dropping**:
 - ▶ With probability $1 - \eta$: train on the real label $y \Rightarrow$ the network learns $u_t^{\text{target}}(x | y)$
 - ▶ With probability η : replace y with a special “no label” token $\emptyset \Rightarrow$ the network learns $u_t^{\text{target}}(x)$
- The **CFG conditional flow matching objective**:

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \left[\|u_t^\theta(x | y) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.17)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$, and y is replaced by $\emptyset$ with prob. η

- At inference: $\tilde{u}_t^\theta(x | y) = (1 - w) u_t^\theta(x | \emptyset) + w u_t^\theta(x | y)$

CFG Training Algorithm

Algorithm 5: CFG Training (Gaussian CondOT path)

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

1. **For** each mini-batch **do**:
2. Sample (z, y) from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x = \alpha_t z + \beta_t \epsilon$
6. With probability η : set $y \leftarrow \emptyset$
7. Compute loss $\mathcal{L}(\theta) = \|u_t^\theta(x \mid y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2$
8. Update θ via gradient descent

CFG for Diffusion Models

- The same idea applies to score-based diffusion models
- By Bayes' rule: $\nabla \log p_t(x | y) = \nabla \log p_t(x) + \nabla \log p_t(y | x)$
- For guidance scale $w > 1$, define the **classifier-free guided score**:

$$\tilde{s}_t(x | y) = (1 - w) \nabla \log p_t(x | \emptyset) + w \nabla \log p_t(x | y) \quad (4.18)$$

CFG Training for Diffusion Models

- The **guided conditional score matching** objective:

$$\mathcal{L}_{\text{CSM}}^{\text{guided}}(\theta) = \mathbb{E}_{\square} \left[\|s_t^\theta(x | y) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.19)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$

- The **CFG conditional score matching** objective (with label dropout):

$$\mathcal{L}_{\text{CSM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \left[\|s_t^\theta(x | y) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.20)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$, replace $y = \emptyset$ with prob. η

- At inference, combine both networks and sample via:

$$\tilde{s}_t^\theta(x | y) = (1 - w) s_t^\theta(x | \emptyset) + w s_t^\theta(x | y)$$

$$\tilde{u}_t^\theta(x | y) = (1 - w) u_t^\theta(x | \emptyset) + w u_t^\theta(x | y)$$

$$dX_t = \left[\tilde{u}_t^\theta(X_t | y) + \frac{\sigma_t^2}{2} \tilde{s}_t^\theta(X_t | y) \right] dt + \sigma_t dW_t$$

Neural Network Architectures

- The neural network $u_t^\theta(x | y)$ must take 3 inputs: $x \in \mathbb{R}^d$, $y \in \mathcal{Y}$, $t \in [0, 1]$, and output a vector in $\mathbb{R}^d$
- For low-dimensional problems: a **multi-layer perceptron (MLP)** suffices (concatenate x , y , t and pass through)
- For images/videos: specialised architectures are needed
- Two dominant architectures:
 - ① **U-Net**¹²: Convolutional; input and output have image shape
 - ② **Diffusion Transformer (DiT)**¹³: Attention-based; patchify the image and process patches with a transformer

¹²Ronneberger, O., Fischer, P. & Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015, arXiv:1505.04597.

¹³Peebles, W. & Xie, S. *Scalable Diffusion Models with Transformers*, ICCV 2023, arXiv:2212.09748.

Working in Latent Space

- For high-resolution images (1000×1000 pixels $\rightarrow \sim 3$ million dimensions), training a diffusion model in pixel space is prohibitively expensive
- **Latent diffusion models:** Train a (variational) **autoencoder** to compress images into a low-dimensional latent space, then train the diffusion model *in the latent space*
- Pipeline:
 - 1 **Encode:** $z_{\text{latent}} = \text{Encoder}(z_{\text{image}})$ — compress training images
 - 2 **Train:** Learn u_t^θ in the latent space
 - 3 **Sample:** Generate X_1 in latent space, then $z_{\text{image}} = \text{Decoder}(X_1)$
- Nearly all state-of-the-art image and video models use latent diffusion

Encoding the Conditioning Variable

- **Class labels** ($y \in \{0, \dots, N\}$): Learn an embedding vector for each class; these embeddings are parameters of u_t^θ
- **Text prompts**: Use a pre-trained text encoder to map y_{raw} (text) to a continuous embedding $y \in \mathbb{R}^{d_y}$
- **CLIP**¹⁴ (Contrastive Language–Image Pre-training): Trains a shared image–text embedding space. Provides a coarse, global embedding
- For richer conditioning: use a pre-trained language model (e.g. T5) to produce a *sequence* of embeddings, and condition via **cross-attention**
- Modern models combine multiple text encoders to capture both coarse semantics and fine-grained details

¹⁴Radford, A. et al. *Learning Transferable Visual Models From Natural Language Supervision*, ICML 2021, arXiv:2103.00020.

Case Study: Stable Diffusion 3

- **Architecture:** Multi-modal Diffusion Transformer (MM-DiT) — a modified DiT that attends to both image patches and text embeddings
- **Training:** Conditional flow matching (CondOT path), with classifier-free guidance (label dropping)
- **Text conditioning:** Three types of text embeddings (CLIP-G/14, CLIP-L/14, T5-XXL) — combining coarse and fine-grained representations
- **Latent space:** Trained in the latent space of a pre-trained autoencoder
- **Scale:** Largest model has 8 billion parameters
- **Sampling:** 50 Euler steps with CFG weight $w \in [2, 5]$

Summary: Building an Image Generator

Recipe for a Large-Scale Image/Video Generator

- ➊ **Probability path:** Gaussian CondOT $p_t(\cdot | z) = \mathcal{N}(tz, (1 - t)^2 I_d)$
- ➋ **Architecture:** Diffusion Transformer (DiT / MM-DiT)
- ➌ **Training:** Conditional flow matching with CFG (label dropout probability η)
- ➍ **Latent space:** Train in the latent space of a pre-trained autoencoder
- ➎ **Text conditioning:** Pre-trained text encoders (CLIP, T5) injected via cross-attention
- ➏ **Sampling:** Euler ODE with CFG weight $w > 1$

Practical: A Conditional Generative Model for Images

- Work through the following Colab notebook on your own:

https://colab.research.google.com/drive/1wKeiJBN4HS-7dsHQk_zd22Zfoc3ieJn9?usp=sharing

- The notebook implements the ideas from this week on the MNIST dataset: conditional flow matching with Gaussian CondOT paths, training a neural network to approximate the vector field, and sampling via ODE integration

End of Week 5